

ЗАТОНУВШИЙ > Запускать рабочие нагрузки

Планирование модулей Kubernetes с помощью планировщика модулей SUNK

 Copy MCP Server

Планируйте модули Kubernetes вместе с заданиями Slurm с помощью планировщика модулей SUNK

SUNK Pod Scheduler позволяет запускать модули Kubernetes вместе с заданиями Slurm в одном кластере. Вместо того чтобы управлять отдельными кластерами для разных типов рабочих нагрузок, вы можете запускать обучение с помощью Slurm, а логический вывод — с помощью Kubernetes на общих узлах.

Когда модуль назначается SUNK Pod Scheduler, создается задание Slurm-заглушка, чтобы Slurm управлял размещением узлов, выделением графических процессоров и отслеживанием ресурсов для модуля.

На этой странице рассказывается о том, как включить планировщик, настроить для него Pod, выбрать стратегию совместного использования узлов, а также об аннотациях и ограничениях, о которых следует знать. Она предназначена для операторов кластера и владельцев рабочих нагрузок, которые хотят объединить рабочие нагрузки Slurm и Kubernetes в общей инфраструктуре.

Включить планировщик

Прежде чем отправлять поды в планировщик SUNK Pod Scheduler, необходимо включить его в Helm-чарте Slurm и указать, за какими пространствами имен он должен следить.

Задайте `scheduler.enabled` в качестве `true` в значениях Helm-чата Slurm.

Спроси

Настройте область действия планировщика для отслеживания пространств имен, в которых вы создаете свои поды:

- Чтобы отслеживать весь кластер, замените `scheduler.scope.type` на `cluster`.
- Чтобы отслеживать определенные пространства имен, замените `scheduler.scope.type` на `namespace` и укажите `scheduler.scope.namespaces` в виде списка пространств имен, разделенных запятыми. Если `scheduler.scope.namespaces` не указано, планировщик по умолчанию использует пространство имен, в котором он развернут.

Настройте Pod для планирования

Если планировщик включен, каждому Pod, который на него настроен, нужны три элемента конфигурации: имя планировщика, запросы на ресурсы и допустимый льготный период при завершении работы. В следующих разделах описаны все требования.

Задайте имя планировщика

В `спес` вашего пода укажите `schedulerName` в качестве имени планировщика пода SUNK, чтобы Kubernetes направлял под в SUNK, а не к планировщику по умолчанию. Имя по умолчанию — `<namespace>-<releaseName>`, но вы можете задать его вручную с помощью `scheduler.name` в значениях Helm.

Задайте требования к ресурсам

Ваш под должен запрашивать определенные ресурсы процессора и памяти, чтобы Slurm мог сопоставить возможности пода с возможностями узла. Если эти требования к ресурсам отсутствуют или равны нулю, под не будет запланирован.

Установите льготный период для завершения работы

если `--slurm-kill-wait=30s` (по умолчанию), то `terminationGracePeriodSeconds` должно быть меньше 25.

⚠ Значение по умолчанию в Kubernetes `terminationGracePeriodSeconds` составляет 30 секунд, что превышает пороговое значение по умолчанию в 25 секунд. Вы должны явно указать это значение в спецификации Pod. Обычно используется значение `5` — `20`.

Look up the scheduler configuration

If you don't know the scheduler name or the current `--slurm-kill-wait` value, you can read both directly from the running scheduler Pod. Run this command to find both the scheduler name and the kill-wait value:

```
kubectl get pods -l app.kubernetes.io/name=sunk-scheduler -oyaml | yq '.items[
```

Найдите в выводе `--scheduler-name` и `--slurm-kill-wait` :

- `--scheduler-name=tenant-slurm-staging-slurm-scheduler`
- `--slurm-kill-wait=30s`

Example Pod

This example shows a full-node GPU Pod with all three required settings:

```
namespace: tenant-slurm-staging
annotations:
  sunk.coreweave.com/account: root
  sunk.coreweave.com/comment: "A Kubernetes job"
  sunk.coreweave.com/exclusive: "none"
spec:
  schedulerName: tenant-slurm-staging-slurm-scheduler
  containers:
    - name: main
      image: ubuntu
      command:
        - /bin/bash
      args:
        - -c
        - sleep infinity
      resources:
        limits:
          memory: 960Gi
          rdma/ib: "1"
          nvidia.com/gpu: "8"
        requests:
          cpu: "10"
          memory: 10Gi
          nvidia.com/gpu: "8"
  terminationGracePeriodSeconds: 5
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: gpu.nvidia.com/model
                operator: In
                values:
                  - H100_NVLINK_80GB
```

Для GPU Pods установите привязку к узлу для `gpu.nvidia.com/model` , чтобы выбрать нужный тип графического процессора. Значения для процессора, памяти и

Choose a Node sharing strategy

After the Pod has the required fields, decide how it shares Nodes with other workloads. The `sunk.coreweave.com/exclusive` annotation controls this behavior, and the right value depends on your workload:

Workload type	Annotation	Effect
Pod needs the entire Node (all GPUs, or full CPU Node)	<code>exclusive: "none"</code>	Full Node isolation. No other jobs can share the Node.
Multiple Pods or jobs share a Node (partial GPUs, or CPU pool)	<code>exclusive: "user"</code>	Jobs from the same Slurm user can share the Node.
Pod can share a Node with any workload	<code>exclusive: "ok"</code>	Open sharing with any user or account.

Подробные инструкции по каждому сценарию, включая настройку ресурсов и примеры, см. в разделе [Управление ресурсами с помощью планировщика SUNK Pod](#).

Annotations reference

These annotations configure Slurm job parameters for SUNK-scheduled Pods. All annotations are optional.

Annotation	Type	Description	Example
<code>sunk.coreweave.com/account</code>	string	Slurm account for the job. List accounts with <code>sacctmgr list account</code> .	<code>"root"</code>
<code>sunk.coreweave.com/comment</code>	string	Descriptive comment to add to the Slurm job.	<code>"A Kubernetes job"</code>
<code>sunk.coreweave.com/constraints</code>	string	Node constraints for job placement. See the Slurm	

exclusive (V5.7.0+)		<u>Exclusive annotation values.</u>	"user"
sunk.coreweave.com/exclusive (before v5.7.0)	boolean	Enables exclusive mode. See the <u>Slurm exclusive documentation.</u>	"true"
sunk.coreweave.com/excluded-nodes	string	Comma-separated Slurm node names to exclude.	"a100-092-01,a100-092-02"
sunk.coreweave.com/required-nodes	string	Comma-separated Slurm node names required.	"a100-092-04,a100-092-06"
sunk.coreweave.com/partition	string	Slurm partition to submit the job to.	"default"
sunk.coreweave.com/preferences	string	Node preferences. See <u>Slurm preferences documentation.</u>	
sunk.coreweave.com/priority	integer	Job priority value. Must be an integer.	"100"
sunk.coreweave.com/quality	string	Quality of Service level.	"normal"
sunk.coreweave.com/reservation	string	Slurm reservation name.	"test"
sunk.coreweave.com/user-id	integer	User ID to run the job as.	"1001"
sunk.coreweave.com/group-id	integer	Group ID to run the job as.	"1002"
sunk.coreweave.com/current-working-dir	string	Working directory path for the job.	"/path/to/use"
sunk.coreweave.com/timeout	integer	Time limit in minutes for the Slurm placeholder job.	"123"

Time limits

The `sunk.coreweave.com/timeout` annotation sets a time limit (in minutes) on the Slurm placeholder job. The Kubernetes time limit begins when the scheduler places the Pod onto a Node, which includes Pod initialization time before the container starts. This is earlier than Slurm's normal time limit, which begins after the prolog script completes.

Known limitations

Before relying on the SUNK Pod Scheduler for production workloads, review the following constraints so you can plan around them.

- No gang scheduling. The scheduler schedules each Pod as a separate Slurm job. Multi-node PodGroups aren't supported. The scheduler is best suited for single-node workloads such as inference.
- No bin-packing. Slurm doesn't fill partially-used Nodes before moving to idle ones. This can spread Pods across Nodes and lead to GPU fragmentation, especially during scaling.
- Kubernetes scheduling features not supported. PodAffinity, PodAntiAffinity, and topology spread constraints have no effect. Slurm makes all Node placement decisions. Node-level affinity (such as `gpu.nvidia.com/model`) is supported.
- Static CPU allocation causes Node drains. Pods with Guaranteed QoS (CPU requests equal to limits) trigger Kubernetes static CPU pinning, which conflicts with Slurm's CPU accounting and creates resource contention. This leads to Node drains in Slurm. For details and prevention steps, see [Static CPU allocation and the SUNK Pod Scheduler](#).
- Non-SUNK Pods are invisible to Slurm. Slurm can't see Pods scheduled through the standard Kubernetes scheduler. Running non-DaemonSet Pods on Slurm Nodes

kubelet rejects the Pod. Check Pod events with `kubect1 describe pod` if a Pod is stuck.

- Scale-down reconfigure delay. When Nodes are removed from the cluster, there's about a one-minute delay before Slurm's configuration updates. During this window, Slurm may try to schedule work onto Nodes that are being removed.

Next steps

- [Manage resources with the SUNK Pod Scheduler](#) for Node sharing configuration, resource tuning, and GPU allocation guidance.

Last modified on May 28, 2026

Was this page helpful?

 Yes

 No

Tunnel VS Code for development on Slurm

[< Previous](#)

Manage resources with the SUNK Pod Scheduler

[Next >](#)